

Comandos `handleLine()`

Resumen de comandos aceptados por `handleLine()` en el firmware de `EasySIMESP32`.

Uso general

- Entrar en configuración: `#CONFIG`
- Salir y aplicar: `#END`
- Muchos comandos de alta, borrado y edición requieren estar entre `#CONFIG` y `#END`.

Comandos directos

Comando	Requiere `#CONFIG`	Ejemplo	Respuesta esperada
---	---	---	---
`PING`	No	`PING`	`PONG`
`#BOARD?`	No	`#BOARD?`	`BOARD ...`
`#DUMP`	No	`#DUMP`	`BEGIN CONFIG ... #END ... BEGIN MB ... END MB`
`#CLEAR`	No	`#CLEAR`	`[OK] EEPROM borrada correctamente.`
`#NVSWIPE`	No	`#NVSWIPE`	borra NVS y reinicia
`#CONFIG`	No	`#CONFIG`	`[OK] MODO CONFIG ACTIVADO`
`#END`	Sí, para cerrar	`#END`	`[OK] MODO CONFIG DESACTIVADO` y reinicio o `#READY`

IO en vivo

Comando	Requiere `#CONFIG`	Ejemplo	Respuesta esperada
---	---	---	---
`IO.WATCH <pin> <kind> ON`	No	`IO.WATCH ADS0 POT ON`	`IO.WATCH OK ...` + `IO.STATE ...`
`IO.WATCH <pin> <kind> OFF`	No	`IO.WATCH ADS0 POT OFF`	`IO.WATCH OK ... OFF`
`IO.READ <pin> <kind>`	No	`IO.READ 12 BUTTON`	`IO.STATE ...`
`IO.WRITE <pin> <0/1>`	No	`IO.WRITE 7 1`	`IO.STATE 7 OUTPUT 1`

Notas:

- Para potenciómetros ADS usa `ADS0`, `ADS1`, etc.
- `kind` puede ser `BUTTON`, `SWITCH`, `OUTPUT`, `POT` o `SELECTOR`.

Red y Ethernet

Solo disponibles si el firmware se compila con Ethernet.

Comando	Requiere `#CONFIG`	Ejemplo	Respuesta esperada
---	---	---	---
`ETH.STATUS`	No	`ETH.STATUS`	estado Ethernet
`ETH.OUT ON/OFF`	No	`ETH.OUT ON`	`OK ETH.OUT ON`
`DISCOVER.SETIP <ip>`	No	`DISCOVER.SETIP 192.168.1.50`	`OK SERVER.IP ...`
`ETH.SERVER <ip>`	No	`ETH.SERVER 192.168.1.50`	`OK SERVER.IP ...`
`ETH.SETIP <ip>`	No	`ETH.SETIP 192.168.1.177`	guarda IP y reinicia

Alta legacy de elementos

Requieren `#CONFIG`.

Comando	Ejemplo	Respuesta esperada
---	---	---
`ADD BUTTON ...`	`ADD BUTTON 12 pzb_wachsam 0 1`	`[OK] Entrada añadida...`
`ADD SWITCH ...`	`ADD SWITCH 13 lzb_on 0 1`	`[OK] Entrada añadida...`
`ADD OUTPUT ...`	`ADD OUTPUT 14 cab_light 0 1`	`[OK] Entrada añadida...`
`ADD POT ...`	`ADD POT ADS0 throttle 0 100`	`[OK] Entrada añadida...`
`ADD SELECTOR ...`	`ADD SELECTOR 10 reverser -1 0`	`[OK] SELECTOR ADD ...`
`ADD ... CANx:y ...`	`ADD BUTTON CAN1:0 pzb 0 1`	`[OK] CAN BUTTON ...`

Configuración de POT (`CFG`)

Requieren `#CONFIG`.

Comando	Ejemplo	Respuesta esperada
---	---	---
`CFG <pin> SCALE ...`	`CFG ADS0 SCALE 0 32767 0 100`	`CFG ACTUALIZADO...`
`CFG <pin> FORMAT INT/FLOAT`	`CFG ADS0 FORMAT INT`	`CFG ACTUALIZADO...`
`CFG <pin> SMOOTH <v>`	`CFG ADS0 SMOOTH 0.10`	`CFG ACTUALIZADO...`
`CFG <pin> MODE CONTINUO`	`CFG ADS0 MODE CONTINUO`	`CFG ACTUALIZADO...`

```
| `CFG <pin> MODE CAMBIO` | `CFG ADS0 MODE CAMBIO` | `CFG ACTUALIZADO...` |
| `CFG <pin> MODE INTERVALO <ms>` | `CFG ADS0 MODE INTERVALO 200` | `CFG ACTUALIZADO...` |
| `CFG <pin> THRESH <v>` | `CFG ADS0 THRESH 0.02` | `THRESH OK en ADS0 = ...` |
```

Selectores (`SEL.\*`)

```
| Comando | Requiere `#CONFIG` | Ejemplo | Respuesta esperada |
|---|---|---|---|
| `SEL.ADD <name> <pin> <val>` | Sí | `SEL.ADD reverser 10 -1` | `[OK] OK SEL.ADD ...` |
| `SEL.DELPIN <pin>` | Sí | `SEL.DELPIN 10` | `[OK] OK SEL.DELPIN` |
| `SEL.CLEAR` | Sí | `SEL.CLEAR` | `[OK] OK SEL.CLEAR` |
| `SEL.DUMP` | No | `SEL.DUMP` | `BEGIN SEL ... END SEL` |
```

Renombrado y borrado

```
| Comando | Requiere `#CONFIG` | Ejemplo | Respuesta esperada |
|---|---|---|---|
| `RENAME.PIN <pin> <name>` | Sí | `RENAME.PIN ADS0 throttle_axis` | `[OK] OK RENAME.PIN ...` |
| `RENAME.NAME <old> <new>` | Sí | `RENAME.NAME reverser reversor` | `[OK] OK RENAME.NAME ...` |
| `#DELETEPIN <pin>` | Sí | `#DELETEPIN ADS0` | `DELETED_PIN ADS0` |
| `#SCANPINS` | Sí | `#SCANPINS` | pines MCP detectados |
```

POT split

Requieren `#CONFIG`.

```
| Comando | Ejemplo | Respuesta esperada |
|---|---|---|
| `POT.SPLIT <pin> OFF` | `POT.SPLIT ADS0 OFF` | `[OK] OK POT.SPLIT ADS0 OFF` |
| `POT.SPLIT <pin> DUAL` | `POT.SPLIT ADS0 DUAL` | `[OK] OK POT.SPLIT ADS0 DUAL` |
| `POT.SPLIT <pin> SIGNED` | `POT.SPLIT ADS0 SIGNED` | `[OK] OK POT.SPLIT ADS0 SIGNED` |
| `POT.SPLIT <pin> CENTERED` | `POT.SPLIT ADS0 CENTERED` | `[OK] OK POT.SPLIT ADS0 CENTERED` |
| `POT.SPLIT.DB <pin> <v>` | `POT.SPLIT.DB ADS0 0.02` | `[OK] OK POT.SPLIT.DB ...` |
| `POT.SPLIT.CBIAS <pin> <v>` | `POT.SPLIT.CBIAS ADS0 0.50` | `[OK] OK POT.SPLIT.CBIAS ...` |
| `POT.SPLIT.TAG <pin> <tag>` | `POT.SPLIT.TAG ADS0 train_brake` | `[OK] OK POT.SPLIT.TAG ...` |
| `POT.SPLIT.TAGS <pin> <fwd> <back>` | `POT.SPLIT.TAGS ADS0 throttle brake` | `[OK] OK POT.SPLIT.TAGS ...` |
```

NOTCH

Requieren `#CONFIG`.

```
| Comando | Ejemplo | Respuesta esperada |
|---|---|---|
| `NOTCH DUMPALL` | `NOTCH DUMPALL` | lista todas las muescas |
| `NOTCH SAVEALL` | `NOTCH SAVEALL` | `OK NOTCH SAVEALL` |
| `NOTCH LOADALL` | `NOTCH LOADALL` | `OK NOTCH LOADALL` |
| `NOTCH PARTIAL <pin> ON/OFF` | `NOTCH PARTIAL ADS0 ON` | `OK NOTCH PARTIAL ...` |
| `NOTCH SNAPWIN <pin> <v>` | `NOTCH SNAPWIN ADS0 0.03` | `OK NOTCH SNAPWIN ...` |
| `NOTCH RAW <pin>` | `NOTCH RAW ADS0` | `OK NOTCH RAW ...` |
| `NOTCH CLEAR <pin>` | `NOTCH CLEAR ADS0` | `OK NOTCH CLEAR ...` |
| `NOTCH ADD <here> <val>` | `NOTCH ADD ADS0 25` | `OK NOTCH ADD ...` |
| `NOTCH ADDHERE <pin> <val>` | `NOTCH ADDHERE ADS0 25` | `OK NOTCH ADDHERE ...` |
| `NOTCH CAP <pin> <idx>` | `NOTCH CAP ADS0 0` | `OK NOTCH CAP ...` |
| `NOTCH CENT <pin> <idx> <raw>` | `NOTCH CENT ADS0 0 15432` | `OK NOTCH CENT ...` |
| `NOTCH VAL <pin> <idx> <val>` | `NOTCH VAL ADS0 0 25` | `OK NOTCH VAL ...` |
| `NOTCH HYST <pin> <v>` | `NOTCH HYST ADS0 0.05` | `OK NOTCH HYST ...` |
| `NOTCH DUMP <pin>` | `NOTCH DUMP ADS0` | dump de ese pot |
| `NOTCH SAVE <pin>` | `NOTCH SAVE ADS0` | `OK NOTCH SAVE ...` |
| `NOTCH DEL <pin>` | `NOTCH DEL ADS0` | `OK NOTCH DEL ...` |
| `NOTCH HYBRIDSEG <pin> <a> <b>` | `NOTCH HYBRIDSEG ADS0 1 3` | `OK NOTCH HYBRIDSEG ...` |
| `NOTCH HYBRIDSEG <pin> OFF` | `NOTCH HYBRIDSEG ADS0 OFF` | `OK NOTCH HYBRIDSEG ... OFF` |
```

Modbus (`MB.\*`)

```
| Comando | Requiere `#CONFIG` | Ejemplo | Respuesta esperada |
|---|---|---|---|
| `MB.PINGIP <ip> [port]` | No | `MB.PINGIP 192.168.1.20 502` | `[OK] MB.PINGIP OK ...` |
| `MB.ADDDEV ...` | No | `MB.ADDDEV 1 TCP 192.168.1.20 502 1` | `[OK] MB.ADDDEV TCP OK` |
| `MB.ADDDEV ... RTU ...` | No | `MB.ADDDEV 2 RTU 3` | `[OK] MB.ADDDEV RTU OK` |
| `MB.DELDEV <id>` | No | `MB.DELDEV 1` | `[OK] MB.DELDEV OK` |
```

```
| `MB.LSDEV` | No | `MB.LSDEV` | lista dispositivos |
| `MB.PING <id>` | No | `MB.PING 1` | `[OK] MB.PING OK ...` |
| `MB.ADDIN ...` | No | `MB.ADDIN 1 HREG 0 2 speed 200 1.0 0.0` | `[OK] MB.ADDIN OK` |
| `MB.ADDOUT ...` | No | `MB.ADDOUT 1 COIL 10 1 lamp` | `[OK] MB.ADDOUT OK` |
| `MB.DELTAG <name>` | No | `MB.DELTAG speed` | `[OK] MB.DELTAG OK` |
| `MB.LSTAG` | No | `MB.LSTAG` | lista tags |
| `MB.DUMP` | No | `MB.DUMP` | `BEGIN MB ... END MB` |
| `MB.READ <name>` | No | `MB.READ speed` | valor leído |
| `MB.SET <name> ...` | No | `MB.SET lamp 1` | `[OK] MB.SET OK ...` |
| `MB.SAVE` | No | `MB.SAVE` | resumen guardado |
| `MB.LOAD` | No | `MB.LOAD` | resumen cargado |
| `MB.CLEAR` | No | `MB.CLEAR` | `[OK] MB.CLEAR OK RAM+EEPROM` |
```

IO lógico Modbus (`MODBUS.\*`)

```
| Comando | Requiere `#CONFIG` | Ejemplo | Respuesta esperada |
|---|---|---|---|
| `MODBUS.ADD ...` | No | `MODBUS.ADD lamp 1 COIL 10 1` | `[OK] MODBUS.ADD OK` |
| `MODBUS.DEL <name>` | No | `MODBUS.DEL lamp` | `[OK] MODBUS.DEL OK` |
| `MODBUS.LS` | No | `MODBUS.LS` | lista IO modbus |
| `MODBUS.READ <name>` | No | `MODBUS.READ lamp` | delega a `MB.READ` |
| `MODBUS.SET <name> ...` | No | `MODBUS.SET lamp 1` | delega a `MB.SET` |
```

Entrada por nombre

También acepta `clave=valor` y lo enruta a outputs locales o tags Modbus.

```
| Comando | Requiere `#CONFIG` | Ejemplo | Respuesta esperada |
|---|---|---|---|
| `clave=valor` | No | `cab_light=1` | `[OK] ACK: cab_light=1` |
| `tag_modbus=valor` | No | `lamp=1` | escribe vía `MB.SET` |
| `tag_modbus=csv` | No | `setpoint=120,45` | escribe varios registros |
```

Referencias de código

- Router principal: src/main.cpp
- Handler Modbus: include/modbus.h